

← Go Back

Hardware

MKR WiFi 1010

Tutorials

Debugging SAM-Based
Arduino® Boards with
Atmel-ICE

Accessing the Built-in RGB
LED on the MKR WiFi 1010

How to Connect Sensors
to the MKR WiFi 1010

Connecting MKR WiFi 1010
to a Wi-Fi Network

MKR WiFi 1010 Bluetooth®
Low Energy

Host a Web Server on the
MKR WiFi 1010

MKR WiFi 1010 Battery
Application Note

Using the Segger J-Link
Debugger with the MKR
Boards

Sending Data over MQTT

Serial to OLED with MKR
WiFi 1010

Powering MKR WiFi 1010
with Batteries

RTC (Real Time Clock) with
MKR WiFi 1010 and OLED
Display

Scanning Networks with
MKR WiFi 1010

Securely Connecting an
Arduino MKR WiFi 1010 to
AWS IoT Core

Web Server Access Point
(AP) Mode with MKR WiFi
1010

Home / Hardware / MKR WiFi 1010 /
How to Connect Sensors to the MKR
WiFi 1010

How to Connect Sensors to the MKR WiFi 1010

Learn how to connect different types of
sensors to the MKR WiFi 1010 board.

Author · Simone Last
Majocchi revision · 07/17/2024

Introduction

In this tutorial we will learn how to
connect different sensors to our
Arduino MKR WiFi 1010 board to
achieve interaction and sensing of
the environment. There are several
types of sensors but we will go
through the three main types,
giving you the basic knowledge to
deal with each type.

The types we will cover are analog,
digital and I2C digital sensors. With
the term sensor we define circuits
and devices that allow our Arduino
board to get an input of some
type, according to a change in the
surroundings.

Goals

Learn three main types of
sensors.

Learn how to get raw data
from sensors.

Use the JWA BME280 library.

ON THIS PAGE

Introduction

Goals

Hardware & Software Needed

Sensor Types +

Creating the Programs +

Conclusion

Hardware & Software Needed

Arduino IDE ([online](#) or [offline](#)).

Arduino MKR WiFi 1010 ([link to store](#)).

Micro USB cable

10k ohm photoresistor

10k ohm resistor

Pushbutton

I2C BME280 pressure, humidity and temperature sensor

Breadboard

Jumper wires

Sensor Types

We will begin by taking a look at the different type of sensors we will be covering in this tutorial.

Digital

The first type of sensor, the simplest one, is basically a contact that can be open and closed at will by some mechanical action. A pushbutton, a microswitch, a magnetic reed are examples of such sensor and they can all be read by Arduino, using the `digitalRead()` function. This type of sensor returns either a HIGH (3.3V) or a LOW (0V) value when IDLE or ACTIVE. Depending on the wiring, this sensor can produce one of these values in the idle status and we will see how to

What is important to remember is that the inputs can't be left "floating" (unconnected) and read reliably: if your pushbutton or sensor is connected and normally open, the reading could pick a random value of High or Low. To avoid these uncertainties it is necessary to use a pullup or pulldown resistor: a high value resistor that keeps the input at a known level when the sensor circuit is open.

To make things easier, the microcontroller has an internal pullup resistor that can be enabled or disabled via software. With a NO (Normally Open) pushbutton we should connect it to ground and enable the pullup resistor with the `pinMode(pin, INPUT_PULLUP)`. At reset or powerup the pullup resistors are disabled.

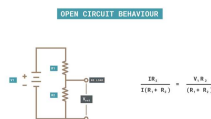
Analog

The analog sensors offer a continuous range of values in an interval, depending on their type. A photoresistor is a passive component that varies its resistance from as low as 4k ohm in full light, down to 30k ohm in darkness, while a moisture sensor reads the amount of water in soil returning a voltage that is proportional to the water content.

In all these cases we have to take in account some details: the sensor can be active or passive and in both cases we use the Arduino circuitry to turn a value that varies in an **analog** fashion

limited number of steps to cover the whole range available. The conversion is made by a special circuitry that quantizes the analog value to the nearest digital step available with 0 or 0v, corresponding to lowest and 1023 or 3.3V, to the highest. The circuit reads a voltage, but this voltage can be a consequence of a resistance by a passive sensor, or a real voltage generated by an active sensor (a solar cell, a piezo trigger, a current sensor). *{/ This last sentence is a little confusing /}*

If a voltage is the consequence of a resistance of some passive component (soil moisture, photoresistor) we need to consider that the voltage read is proportional to the current that flows into the resistor. Arduino analog inputs have a very high input resistance and therefore they do not represent a load. In other words with a negligible flow of current, the voltage read at the analog input will almost always be the same to the sensor. To get a variation in the reading we need to create a resistive divider. It is a simple circuit where we have two resistors connected in series, as shown in the graphic.



Open circuit behavior.

We have the open circuit formula because the Arduino Analog Input

have a fixed value as well, but if we have our photoresistor as one of the two resistors, we will have a varying V_{out} according to the change of illumination on the sensor. The variation direction depends on where you put the photoresistor in this circuit: R1 to get a higher value from

`analogRead()` when light goes up, R2 to get a lower value from `analogRead()` when light goes up.

In any case, we can't have the full range of 1023 steps from this circuit because the photoresistor has a minimum and a maximum resistance that goes into our equation together with the fixed resistor. It is easy to see that we can get very close to either 0V or 3.3V adjusting the value of the resistor, but the range will be biased towards one side or the other.

An active sensor can solve this using some circuitry that defines a minimum and a maximum and that will translate to the full range from 0V to 3.3V. An important reminder is that our MKR WiFi 1010 is a 3.3V microcontroller and any input can tolerate up to this voltage. Any level of 5V or more to any of its input will damage the microcontroller.

I2C

These sensors use a serial two wire interface that allows the coexistence of several devices on the same bus. Each device is

/ SCL) and each has a unique 7 bit address that is used to communicate with it. The communication is initiated by the controlling device (the Arduino) and the device with the corresponding address executes the command received. Each device has its own set of commands and functionalities, this is why we usually have a library for each device together with the standard wire library that takes care of I2C protocol.

The nice thing about I2C is that we can also have actuators, like an LCD screen, on the bus making the wiring of an entire device with sensors and actuators a quite simple matter: just four wires going from one device to the next one (VCC, GND, SDA, SCL). Some devices have additional connections for special functions.

The number of addresses available for I2C devices is 127 (7 bit) and 0 is the "general call" address. As you may imagine, the number of devices that use I2C are more than 127 and there are many cases where you need to use more than one device of the same type in the same circuit. Addressing, offers some flexibility and it is possible to change the address of some devices by hardware or software.

An alternative is to use I2C on different buses (connections) with a circuit that allows us to select the proper bus.

Last, but not least important, every Arduino can be either the controller or the controlled device

that takes care of I2C communication is **Wire** and supports both roles.

Creating the Programs

First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager....** Here we need to look for the **Arduino SAMD boards (32-bits Arm® Cortex®-M0+)** and install it.

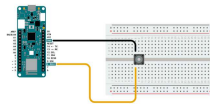
Digital Sensor Example

In this example we connect a pushbutton to the board, on D8 and to the GND. To avoid a floating value on our digital pin when the pushbutton is not pressed, we use `pinMode(D8, INPUT_PULLUP)` that activates the internal pullup resistor and therefore reads a 1 when the contact is open. It is worth noting that this solution gives a 0 when the button is pressed and usually this value is associated with the logic state **false**.

If you want to reverse the situation, with a 1 read when the pushbutton is pressed, you need to connect a high value resistor (10k ohm is fine) to ground and to the digital pin to which you

pushbutton is open the digital pin reads the ground level through the resistor (a pull down resistor) and when the pushbutton closes, the VCC will show up at the digital pin.

Here is the wiring diagram for the internal pullup solution:



Circuit for internal pullup solution.

You can find the code in the snippet below:

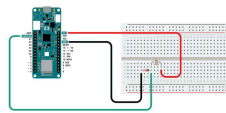
```
1 void setup() {  
2   pinMode(8, INPUT_PULLUP)  
3   pinMode(LED_BUILTIN, OUTPUT)  
4 }  
5 void loop() {  
6   digitalWrite(LED_BUILTIN, digitalRead(8))  
7   // we echo the status  
8 }
```

Analog Sensor Example

The Light Dependent Resistor (LDR) is a passive component that varies its resistance according to the light hitting its active area (the brown material between the conductive fingers - cadmium sulphide). It is a passive component as it does not amplify or add energy to the circuit. For instance, we can say that it works as a potentiometer with the knob

In this example we create a resistor divider with the LDR and a 10K ohm resistor. The LDR is connected to the VCC pin (3.3V) on one side and A0 on the other. The 10K ohm is connected to A0 and GND.

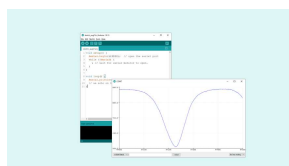
Here is the wiring diagram for connecting the LDR:



Circuit for connecting an LDR.

```
1 void setup() {  
2   Serial.begin(115200);  
3   while (!Serial) {  
4     ; // wait for serial  
5   }  
6 }  
7  
8 void loop() {  
9   Serial.println(analog  
10  // we echo on the ser  
11 }
```

The code is simplified to the essential and the best results are achieved by opening the Serial Plotter. The graph will float and clearly show the variation of the resistance when you move your hand in front of it.



I2C Sensor Example

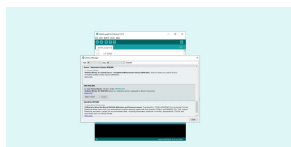
In the sea of I2C sensors we chose a classic to provide a simple but effective example. The choice fell on **BME280 by Bosch Sensortec**. This sensor measures temperature, humidity and pressure, allowing you to calculate some theoretical altitude if you know the sea level pressure of your location.

It is compatible with the MKR board that works with 3.3V and most of the modules you find on the market have support circuitry to support 5V and 3.3V.

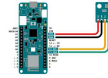
The VCC and GND go to MKR VCC and GND pins, while SDA (Data) and SCL (Clock) correspond to the pins 11 and 12. These pins are the default ones, but it is possible to reprogram the SAMD21 to use different pins. You can find more information at [this link](#).

The four wires are all it is needed for our sensor. If you want to connect other sensors, you just daisy chain them bringing the same four wires from this sensor to the next device.

The BME280, as almost all other I2C devices, requires a BME280 library to be used. You can easily install this in the library manager.



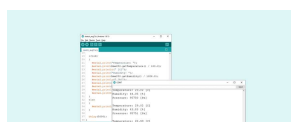
Here is the wiring diagram for connecting the BME280 sensor:



Connecting the BME280 sensor.

```
1  #include <Bme280Bosch>
2
3  Bme280BoschWrapper bme280;
4
5  void setup() {
6    Serial.begin(9600);
7    Serial.println("BME280");
8
9    while(!bme280.begin())
10   {
11     Serial.println("Could not find BME280 sensor!");
12     delay(1000);
13   }
14 }
15
16 void loop() {
17   bool ok = bme280.measure();
18
19   if(ok)
20   {
21     Serial.print("Temp: ");
22     Serial.print(bme280.temperature);
23     Serial.println("C");
24     Serial.print("Hum: ");
25     Serial.print(bme280.humidity);
26     Serial.println("%");
27     Serial.print("Pres: ");
28     Serial.print(bme280.pressure);
29     Serial.println("hPa");
30   }
31   delay(1000);
32 }
```

The output of this example should look, with different values like this:



Printing the BME280's
values in the Serial
Monitor.

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

We have not updated the latest firmware for the board.

We have not installed the Board Package required for the board.

We have not installed the **JWA BME280** library.

We have selected the wrong port to upload: depending on what computer we use, sometimes the board is duplicated. By simply restarting the editor, this issue can be solved.

Conclusion

Sensors are a very important part of physical computing and the Arduino boards are made to support a huge variety of them. In this tutorials we gave you the right clues to connect the main types and you can find a lot more of specific examples for I2C sensors on our Project Hub.

Suggest changes **Need support?**

The content on this page is licensed under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). [Help Center](#) [Ask the Community](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-ShareAlike 4.0 International License](#).

through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

[Discord](#)

[Attribution-Share Alike 4.0](#) license.

Was this article helpful?

